

Kimi K2、Agentic LLM 与站在无限的开端

Zhang Xiaojun 对话月之暗面创始人杨植麟

基于 Zhang Xiaojun Podcast 公开视频、GPU 转写稿、章节结构与自制概念图整理

2025-08-27



视频标题: 113. 和杨植麟时隔 1 年的对话: K2、Agentic LLM、缸中之脑和“站在无限的开端”

视频频道: Zhang Xiaojun Podcast

视频时长: 01:41:14

视频链接: <https://www.youtube.com/watch?v=ouG6jrkeCrc>

素材说明: YouTube 无可用字幕; 正文基于 faster-whisper large-v3 CUDA 转写、视频章节、视频描述、封面和自制教学图整理。固定封面视频不在正文重复使用。

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 导读: K2 为什么是“缸中之脑”长出手	2
1.1 本章小结	3
2 一座无限的山: 问题不可避免, 但问题可以解决	3
2.1 Test-time scaling: 长思考和 Agent 指向同一方向	4
2.2 L1 到 L5 不一定串行	5
2.3 本章小结	6
3 K2 是乔戈里峰: 基础模型、效率与 Agentic 能力	7
3.1 Token efficiency: 同样数据, 脑子长得更多	7
3.2 Agentic 模型的泛化挑战	8
3.3 本章小结	9
4 K2 技术决策蓝图: 四个目标如何互相牵制	9
4.1 训练效率与产品能力的循环	10
4.2 本章小结	10
5 既简单又复杂的系统: 开源、数据墙与架构取舍	11
5.1 Scaling Law、数据墙与 FLOPs	11
5.2 Linear Attention 与“智商”风险	12
5.3 基座模型公司与 Agent 产品公司的边界	13
5.4 本章小结	15
6 在自己的故事里面: RL 管理、恐惧与文明放大器	15
6.1 复杂性与故事	16
6.2 恐惧与当前一步	16
6.3 本章小结	16
7 总结与延伸	17
7.1 关键 takeaways	17
7.2 与前后几集的关系	18
7.3 开放问题	18
7.4 拓展阅读	19

1 导读：K2 为什么是“缸中之脑”长出手

本节先建立整期的技术主线。Kimi K2 是一个基于 MoE 架构的开源编程和 Agentic 大语言模型。访谈里最有力的比喻是“缸中之脑”：只会思考和输出文本的模型像被放在鱼缸里的大脑，和外部世界没有真实连接；而当模型具备编程能力、多轮工具使用能力和外部反馈时，它就开始长出“手”，能操控数字世界。

这期要理解四件事。第一，K2 为什么把基础模型、token efficiency、Agentic 能力和开源放在一起。第二，test-time scaling 为什么同时指向长思考 RL 和 Agent RL。第三，数据墙、FLOPs scaling、Linear Attention、长上下文架构和模型“智商”之间怎样取舍。第四，杨植麟如何把技术判断、商业模式和创始人心态放进“站在无限的开端”这个叙事里。



图 1：从缸中之脑到 Agent：编程能力和工具使用让模型开始操控数字世界。自制概念图，依据 00:01:19–00:24:58 对谈内容整理。

读图：为什么“手”是关键

模型会思考和会行动是两件事。Code、工具调用、API、文件系统和反馈环境，让模型的推理可以改变外部状态。Agentic LLM 的关键不只是多说几步，而是多轮使用工具并根据结果修正。

本期核心命题

K2 的意义不只是“又一个强模型”，而是把基础模型、编程能力、工具使用、开源生态和 test-time scaling 放进同一个系统：让模型从封闭认知系统走向可行动的数字智能体。

本期系统总览

层级	核心问题	访谈里的答案
哲学层	为什么继续攀登?	问题不可避免, 但问题可以解决。
模型层	K2 怎样成为强底座?	MoE、token efficiency、Muon、Rephrase。
行动层	模型怎样长出手?	编程能力、多轮工具使用、Agentic 泛化。
生态层	为什么开源?	让基础模型能力被外部验证和生态放大。
商业层	怎么赚钱、边界在哪?	API、Agent 产品、基座模型公司 和应用公司重新划线。
组织层	创始人如何承受复杂性?	用 RL 式反馈管理, 但警惕被指标 hack。

术语消化: 本期关键词

术语	含义	为什么重要
MoE	Mixture of Experts, 按 token/任务路由到不同专家	提高参数容量,同时控制每次激活计算。
Agentic LLM	能多轮使用工具、观察反馈并完成任务的语言模型	从聊天走向数字世界行动。
Test-time scaling	推理时投入更多计算、思考或工具调用	让模型在测试阶段继续扩展能力。
Token efficiency	每个训练 token 带来的能力收益	数据墙时代, 喂同样数据要“脑子长得更多”。
Muon	访谈中提到的优化器方向	可能提升训练效率,但训练稳定性有挑战。

1.1 本章小结

EP113 是 K2 发布后的技术判断访谈。它把 Agentic LLM 放在模型公司战略、开源、训练效率、架构取舍和创始人心理之中, 适合与 EP119 的 Attention 架构综述、EP115 的 Agent 理论框架对读。

2 一座无限的山: 问题不可避免, 但问题可以解决

本章从《The Beginning of Infinity》开始。杨植麟说, 问题是不可避免的, 但问题是可以解决的; 也许这座雪山没有尽头, 但他希望它一直没有尽头。这个比喻很适合模型公司: 每次模型能力提高, 都

会解锁新场景，同时展开新问题。预训练、RLHF、长思考、Agentic 能力、工具使用和评价体系，都像雪山不同高度的路线。

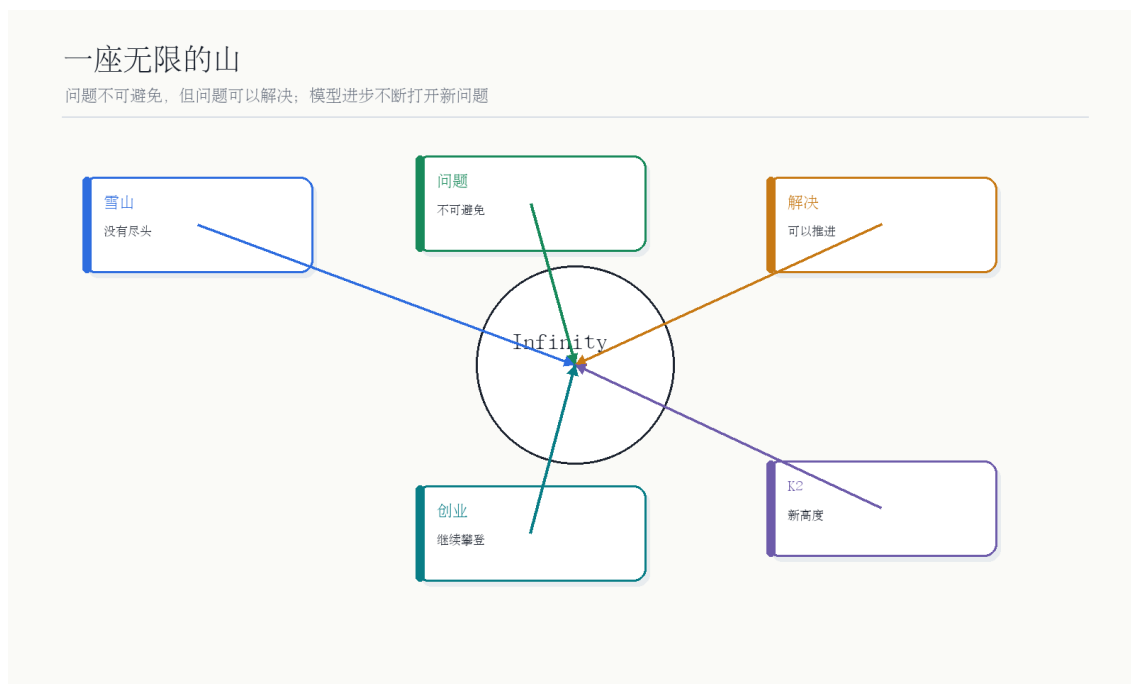


图 2: 一座无限的山：问题不可避免，但问题可以解决；模型进步不断打开新问题。自制概念图，依据 00:00:02–00:05:18 对谈内容整理。

读图：无限不是悲观，而是研究氧气

如果山有尽头，模型公司很快会变成纯工程公司；如果山没有尽头，研究、产品和组织都会持续面对新问题。无限的山意味着长期问题空间仍然存在。

2.1 Test-time scaling: 长思考和 Agent 指向同一方向

上一节讲“问题会继续展开”，本节进入第一个技术判断。杨植麟认为，不管是基于长思考的强化学习，还是 Agent 的强化学习，都指向 test-time scaling。所谓 test-time scaling，就是在推理/测试阶段投入更多计算：更长推理链、更多工具调用、更多验证和更多迭代。



图 3: Test-time Scaling: 长思考 RL 和 Agent RL 都在扩展推理时计算。自制概念图，依据 00:03:38–00:24:58 对谈内容整理。

读图：测试时扩展不是只多输出 token

长思考让模型在内部推理上花更多 token；Agent 让模型通过工具和环境获得外部反馈。二者都在测试阶段花更多计算，只是一个偏内部思考，一个偏外部行动。

Test-time scaling 的直觉公式

$$\text{Capability} \approx f(\text{model}, \text{test-time compute}, \text{feedback})$$

其中，model 是基座能力，test-time compute 是推理阶段额外投入，feedback 是验证、工具返回或环境反馈。Agentic LLM 的关键是把后两者做成系统。

2.2 L1 到 L5 不一定串行

上一节把 test-time scaling 拆成内部思考和外部行动，本节进一步说明这些能力不一定线性爬楼。访谈提到 L1 到 L5 不一定是串行关系，Claude 就是在 reasoning 不一定最强的情况下，在 Agent 上做得很好。这个判断很重要：模型能力发展不是固定阶梯，Reasoning、Agent、工具、产品和反馈可以并行押注。只有当模型参与到开发过程，才可能解锁真正的 Innovator 阶段。

L1 到 L5 不一定串行

Reasoning 与 Agent 可以并行押注，Claude 是典型例子



读图：从左到右看依赖关系；正文负责展开细节，图只保留骨架。

图 4: L1 到 L5 不一定串行：Reasoning 与 Agent 可以并行押注，Claude 是典型例子。自制概念图，依据 00:12:00–00:24:58 对话内容整理。

不要把能力等级当作机械阶梯

如果把 L1 到 L5 理解为必须逐级通关，就会低估产品和系统路线。Agent 能力可能在某些任务上先跑出来，再反过来推动模型训练和工具生态。

L 阶段的教学解释

阶段	直觉	关键变量
L1/L2	聊天、问答、基础 reasoning	模型底座与对齐。
L3	能使用工具完成明确任务	工具接口、任务定义、反馈。
L4	参与开发、创新和复杂问题解决	长程 Agent、验证、代码能力。
L5	更高自主性系统	安全、组织、权限和持续反馈。

2.3 本章小结

“无限的山”给这期提供了精神底色：模型能力每前进一步都会展开新问题；test-time scaling 则给出技术方向：让模型在推理时思考更多、行动更多、获得更多反馈。

3 K2 是乔戈里峰：基础模型、效率与 Agentic 能力

本章进入 K2。杨植麟把 K2 的目标拆成几个部分：第一，希望它是非常好的基础模型；第二，希望最大化使用每一份数据，也就是提高 token efficiency；第三，希望它有好的 Agentic 能力，能从“缸中之脑”走向多轮工具使用；第四，开源让能力进入生态。



图 5: Kimi K2 设计目标：基础模型、token efficiency、Agentic 能力和开源同时成立。自制概念图，依据 00:24:58-00:54:08 对谈内容整理。

读图：K2 不是单点优化

基础模型决定底座，token efficiency 决定同样数据能长出多少能力，Muon 和 Rephrase 是训练效率手段，Agentic 能力决定模型能否使用工具，开源决定生态扩散。

3.1 Token efficiency：同样数据，脑子长得更多

数据墙之后，token efficiency 变得更重要。杨植麟的说法是，喂一样多的数据，希望“脑子”长得更多。K2 会对数据做 Rephrase，也关注 Muon 优化器等方向。这些技术不只是为了 benchmark，而是为了在高质量数据越来越稀缺时，提高每个 token 的训练收益。

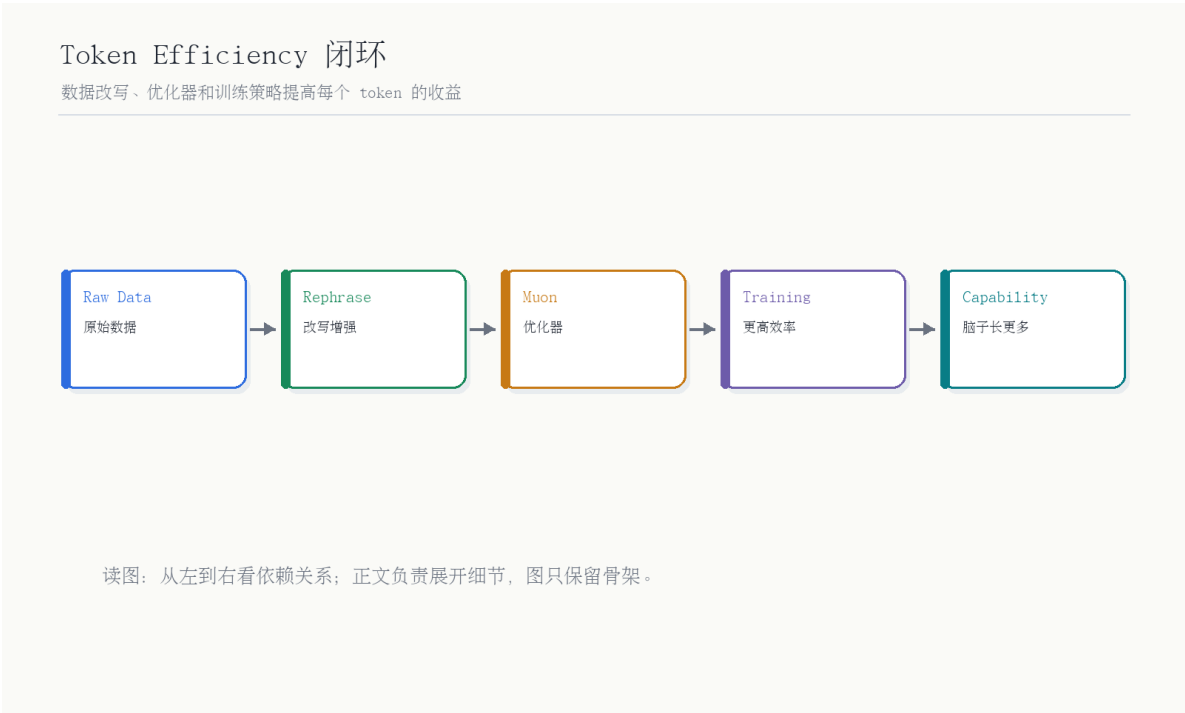


图 6: Token Efficiency 闭环：数据改写、优化器和训练策略提高每个 token 的收益。自制概念图，依据 00:27:28–00:32:44 对话内容整理。

术语消化：训练效率相关概念		
概念	作用	风险
Token efficiency	同样 token 带来更多能力	难以单独度量，常被其他变量混淆。
Rephrase	改写数据以增加表达变化	改写质量差会引入噪声。
Muon	优化器方向，可能提升训练效率	训练可能不稳定，访谈中提到“会炸”。
Data mixture	数据配比和课程	配错会伤害能力或泛化。

3.2 Agentic 模型的泛化挑战

上一节讲 K2 的训练效率，本节转向 Agentic 能力的核心难题：泛化。对于 Agentic 模型，最大挑战是泛化。训练环境里的工具、任务和反馈如果太窄，模型可能只会在熟悉场景里表现好；真正有价值的是遇到新工具、新任务、新环境时仍能多轮探索、调用工具并修正行为。

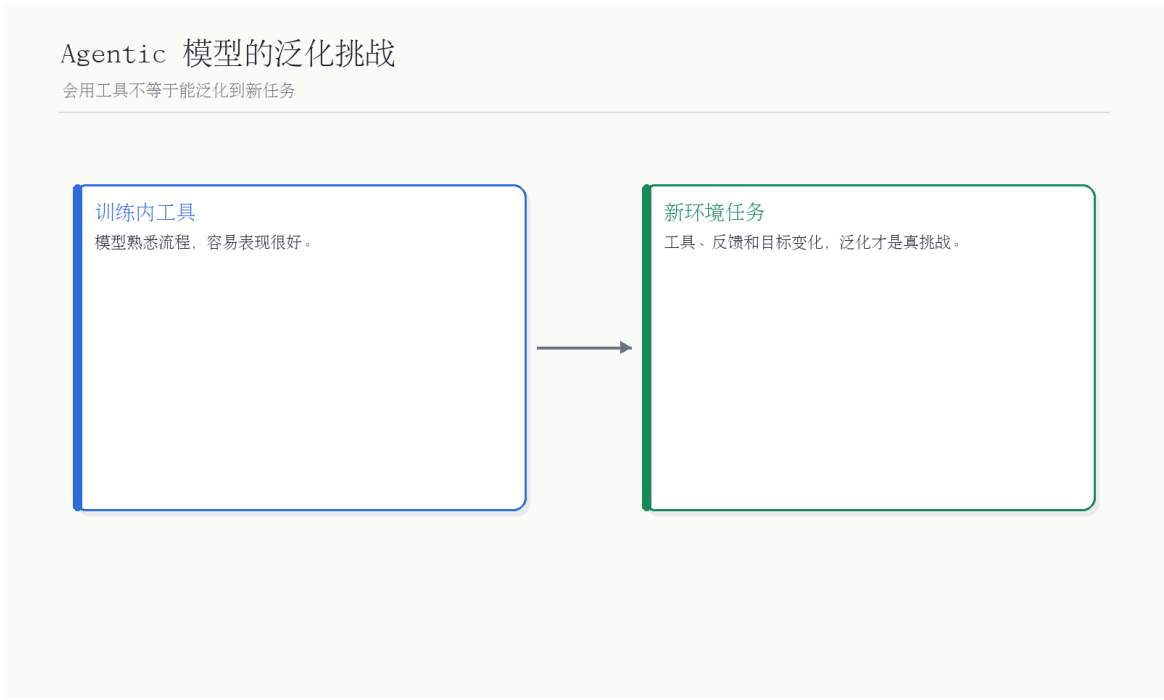


图 7: Agentic 模型的泛化挑战: 会用工具不等于能泛化到新任务。自制概念图, 依据 00:32:50–00:38:00 对谈内容整理。

Agentic 泛化的判断		
一个模型是否真正 Agentic，不是看它是否会调用一个固定工具，而是看它能否在未知任务中理解工具说明、规划多步动作、从反馈中修正，并把能力迁移到新环境。		
Agentic 泛化的三层测试		
层级	测什么	失败样式
工具泛化	新 API、新命令、新文档是否能理解	只会训练中见过的工具。
任务泛化	新目标和长程依赖是否能分解	计划漂亮但无法落地。
反馈泛化	错误返回后能否自我修正	一错到底，或重复同一动作。

3.3 本章小结

K2 的核心是多目标平衡: 强基础模型、训练效率、Agentic 泛化和开源生态必须同时成立。任何一个环节短板，都会限制它从模型变成系统。

4 K2 技术决策蓝图：四个目标如何互相牵制

上一章说明 K2 不是单点优化，本章把这些目标整理成决策蓝图，回答“模型公司到底在同时优化什么”。基础模型、Agentic 能力、开源生态和商业化不是并列口号，而是互相牵制的四个目标。基

基础模型越强，Agent 产品和 API 越有底气；开源越彻底，生态扩散越快，但商业捕获越难；Agentic 能力越强，工具和反馈越重要，训练和评测也越复杂。

K2 四目标矩阵

目标	要优化什么	会牵动什么
Base Model	通用能力、代码能力、推理能力	数据质量、token efficiency、优化器和算力。
Agentic	多轮工具使用、环境反馈、任务泛化	RL、测试时计算、工具协议和评测。
Open Source	外部验证、开发者生态、信任	商业捕获、支持成本和竞品复用。
Business	API、产品、企业和开发者付费	模型边界、Agent 应用边界和成本结构。

为什么这四个目标不能拆开看

如果只有强基座而没有 Agentic 能力，模型仍然像“缸中之脑”；如果只有 Agent 产品而基座不强，泛化会受限；如果开源但没有商业路径，生态难以持续；如果商业化过早，又可能限制研究和开源扩散。

4.1 训练效率与产品能力的循环

四目标矩阵里最容易被低估的是训练效率，本节说明它为什么会传导到产品。Token efficiency 看起来是训练指标，但它会影响产品能力。更高的 token efficiency 意味着同样数据和算力下得到更强底座；更强底座让 Agent 更能处理复杂任务；复杂任务产生更多反馈和使用数据；这些反馈又可能进入下一轮训练和产品迭代。这个循环如果跑起来，模型公司就不只是训练模型，而是在构建能力飞轮。

能力飞轮

环节	作用	风险
训练效率	降低获得能力的成本	优化器或数据策略不稳定。
Agent 产品	让模型进入真实任务	失败率、工具安全和用户信任。
反馈数据	记录真实任务中的成功和失败	噪声大、隐私和选择偏差。
下一轮训练	用反馈修正模型和工具策略	评测不准会放大错误。

4.2 本章小结

K2 的技术决策不是单点最优，而是四目标联合优化。理解这个蓝图，才能理解为什么访谈会在 token efficiency、Agentic 泛化、开源、API 和商业模式之间来回切换。

5 既简单又复杂的系统：开源、数据墙与架构取舍

上一章讲 K2 设计目标，本章看系统层取舍。为什么从闭源转向开源？为什么多模态不损伤“脑子”已经很好？为什么 Scaling Law 遇到数据墙后，FLOPs、RL、反馈和架构效率要重新平衡？这些问题说明，模型公司并不是只调一个旋钮，而是在复杂系统里不断取舍。

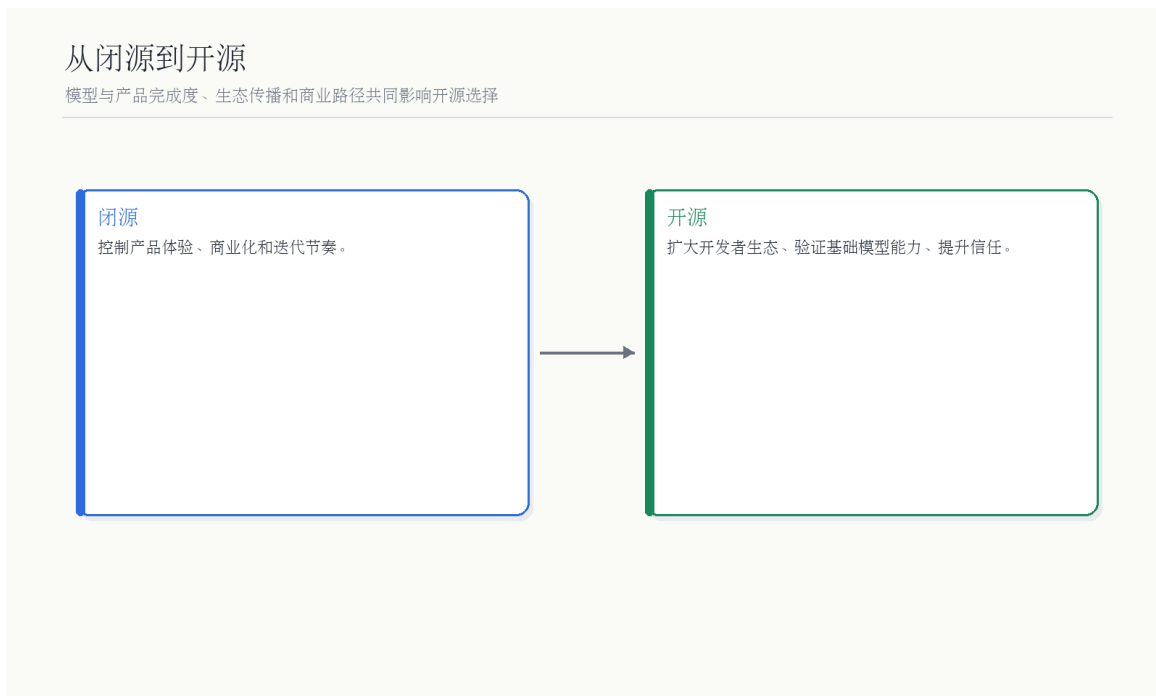


图 8：从闭源到开源：模型与产品完成度、生态传播和商业路径共同影响开源选择。自制概念图，依据 00:54:08–01:05:00 对谈内容整理。

读图：开源是技术选择，也是生态选择

闭源有利于控制产品体验和商业化，开源有利于生态扩散、开发者验证和信任建立。K2 开源意味着它既要证明基础模型能力，也要让 Agentic 生态围绕它生长。

5.1 Scaling Law、数据墙与 FLOPs

上一节讨论开源和生态，本节回到模型训练的硬约束。杨植麟认为 Scaling Law 遇到数据墙是客观事实。高质量数据增长变慢以后，继续 scale 数据不再像过去那么简单；于是 FLOPs scaling、RL、环境 feedback、数据飞轮和架构效率的重要性上升。当前看，基于 FLOPs 的 scaling 仍然有效，但未来平衡点可能变化。

Scaling 与数据墙

数据墙之后，FLOPs、RL、反馈和架构效率重新平衡



读图：从左到右看依赖关系；正文负责展开细节，图只保留骨架。

图 9: Scaling 与数据墙：数据墙之后，FLOPs、RL、反馈和架构效率重新平衡。自制概念图，依据 01:03:00–01:10:00 对谈内容整理。

数据墙不是训练停止

数据墙不是说没有数据了，而是高质量、低噪声、可带来边际能力提升的数据变稀缺。模型公司仍然可以通过更好数据配比、合成数据、RL、工具环境和 test-time scaling 继续推进。

数据墙后的四条路

路线	解决什么	代价
FLOPs scaling	用更多计算继续压 loss	成本高，依赖硬件供给。
Data efficiency	同样数据训练出更多能力	需要更好过滤、改写、配比和优化器。
RL/feedback	从环境反馈中创造新学习信号	reward 噪声和安全问题更难。
Architecture	用新结构降低长上下文/推理成本	可能引入能力 bias。

5.2 Linear Attention 与“智商”风险

数据墙之后，架构效率会变得更诱人，本节讨论这种诱人的风险。访谈提到，很多 long context 架构会影响“智商”；纯粹 Linear Attention 可能因为架构 bias 影响模型能力。这个判断和 EP119 的 Attention 架构综述形成呼应：降低长上下文成本很重要，但如果架构改变牺牲了推理、记忆或表达能力，就可能影响模型上限。

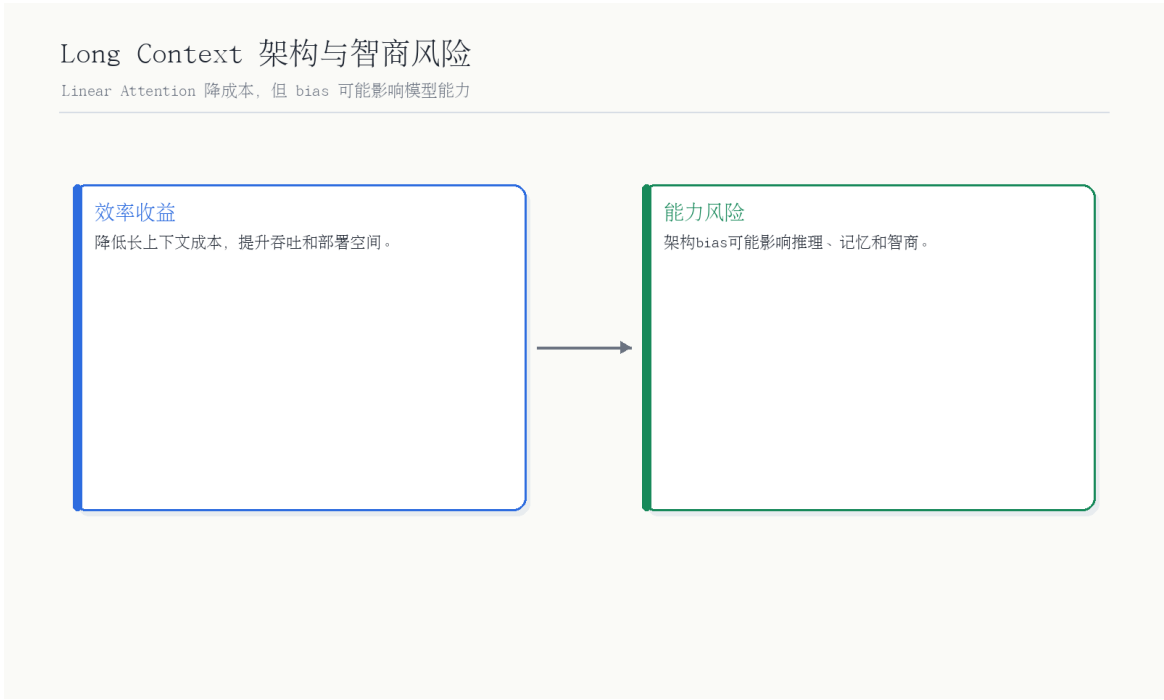


图 10: Long Context 架构与智商风险: Linear Attention 降成本, 但 bias 可能影响模型能力。自制概念图, 依据 01:17:17-01:20:00 对谈内容整理。

术语消化: 架构取舍		
路线	收益	风险
Full Attention	表达力强、经验成熟	长上下文成本高, KV cache 压力大。
Linear Attention	成本低、长序列友好	架构 bias 可能影响能力。
Sparse Attention	降低计算, 保留部分全局性	选择机制和硬件效率复杂。
Hybrid	效率和能力折中	比例、训练稳定和评测更复杂。

5.3 基座模型公司与 Agent 产品公司的边界

前面讨论模型内部的架构取舍, 本节把问题推向产业边界。长期看, 基座模型公司和 Agent 应用公司的边界并不固定。基座模型公司掌握模型能力、训练数据、API 和平台生态; Agent 应用公司掌握场景、工作流、用户入口和反馈数据。谁能把模型能力转成真实任务闭环, 谁就能占据更高价值位置。

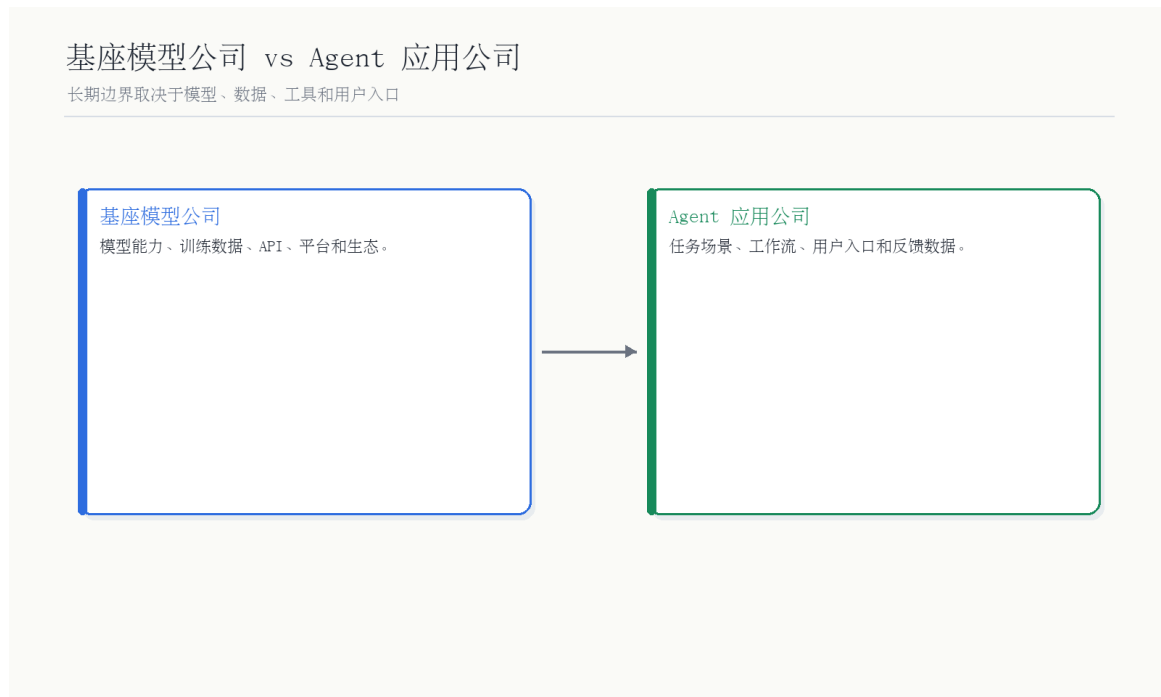


图 11: 基座模型公司 vs Agent 应用公司: 长期边界取决于模型、数据、工具和用户入口。自制概念图, 依据 01:09:00–01:15:00 对谈内容整理。

边界判断

如果 Agent 产品只是一层薄 workflow, 边界会被基座模型公司吞掉; 如果 Agent 产品拥有强场景、强数据回流和高频用户入口, 它就可能反过来定义模型需求。

商业模式对照: API、产品与生态

模式	优势	风险
API	易规模化, 开发者可接入	token 价格竞争, 离用户价值远。
Agent 产品	更接近任务和用户入口	需要产品、工具、反馈和可靠性。
开源生态	扩散快, 信任和开发者参与强	商业捕获更难, 支持成本高。
闭源产品	体验可控, 商业闭环清晰	生态扩散和外部验证受限。

商业模式还有一个成本侧问题: Agentic 任务通常比普通聊天消耗更多推理时计算。一个简单问答可能只需要一次模型调用; 一个复杂 Agent 任务可能需要计划、搜索、工具调用、验证、重试和总结。这样会让“用户价值”和“推理成本”之间的差距变大: 如果任务价值高, Agent 成本可以被吸收; 如果任务只是低价值聊天, 成本很容易压垮毛利。

Agent 成本结构

成本项	来自哪里	降本方式
模型调用	多轮规划、执行、反思	小模型路由、缓存、任务拆分。
工具调用	搜索、代码、浏览器、API	更好的工具协议和失败恢复。
验证成本	测试、检查、人工确认	自动验证器和风险分级。
上下文成本	长历史、文件、代码库、网页	记忆压缩和检索策略。

5.4 本章小结

K2 背后的系统取舍包括开源策略、数据墙、FLOPs scaling、架构效率和产品边界。它不是单点技术竞赛，而是模型公司在复杂系统中的多变量优化。

6 在自己的故事里面：RL 管理、恐惧与文明放大器

本章回到创始人视角。Tim 说要用 RL 的方式管理，而不是只用 SFT。这个比喻很有意思：SFT 像给团队看标准答案，RL 像设定目标和反馈，让团队在探索中学会策略。但 RL 管理的风险是容易被 hack，也就是团队可能优化指标而不优化真实目标。

用 RL 管理，而不是只用 SFT

目标、反馈和奖励会塑造团队，但也容易被 hack



读图：从左到右看依赖关系；正文负责展开细节，图只保留骨架。

图 12: 用 RL 管理，而不是只用 SFT：目标、反馈和奖励会塑造团队，但也容易被 hack。自制概念图，依据 01:25:05–01:34:00 对谈内容整理。

读图：管理隐喻为什么有用

SFT 管理强调示范和标准答案，RL 管理强调目标、反馈和探索。模型训练和组织管理都需要 feedback，但 feedback 设计不好就会被 hack。

6.1 复杂性与故事

上一节讲 RL 管理会塑造团队，本节回到创始人如何理解这种复杂性。杨植麟说很多复杂性是人为强行加上的，实际并没有那么复杂；但创始人仍然要在自己的故事里面感受自己是什么样的人，为什么做这件事。这不是逃离技术，而是承认模型公司是技术系统和叙事系统的叠加：技术路线需要判断，组织需要意义，创始人需要在不确定性里继续行动。

AI 是人类文明的放大器

访谈中提到，Kimi 对“AI 是什么”的回答是：AI 是人类文明的放大器。这个定义把模型从工具提升到文明尺度：它放大知识、生产力、组织能力，也放大人的恐惧、误解和中间状态的争议。

创始人故事里的三种复杂性

复杂性	来源	应对方式
技术复杂性	模型、数据、架构、产品同时变化	回到可验证问题和下一步实验。
组织复杂性	团队、反馈、舆论、管理指标互相影响	用反馈校正，但防止 reward 被 hack。
叙事复杂性	中间状态会被外界误读或批评	在自己的故事里保持行动方向。

6.2 恐惧与当前一步

面对舆论风暴和创业起伏，杨植麟承认肯定有恐惧，但更重要的是关注当前这一步能做什么。这个态度和“无限的山”互相呼应：如果问题不可避免，恐惧也不可避免；但问题可以解决，所以行动要回到下一步。

中间状态一定会被批评

任何复杂系统在中间状态都不完美，也都可能成为批评对象。模型公司尤其如此：能力、产品、开源、商业化、组织和舆论都处在动态平衡中。不能因为中间状态被批评，就停止解决问题。

6.3 本章小结

最后一章把技术判断放回创始人故事。K2 不只是模型发布，也是一次关于无限问题、组织反馈、恐惧管理和文明放大器的自我叙事。

7 总结与延伸

本节把整期压缩成五个结论。第一，Agentic LLM 的关键是从“缸中之脑”走向能使用工具的数字行动者。第二，test-time scaling 同时包括长思考和 Agent 工具反馈。第三，K2 把基础模型、token efficiency、Muon/Rephrase、Agentic 泛化和开源生态放在同一套设计中。第四，数据墙之后，FLOPs、RL、反馈和架构效率会重新平衡，Linear Attention 等低成本路线必须警惕能力 bias。第五，模型公司不仅要解技术题，也要解开源、商业模式、组织反馈和创始人心态题。

关键判断清单		
判断	为什么成立	仍需观察
Agentic 是下一个主线	模型已经能写代码、用工具、长程执行	泛化和可靠性是否能持续提升。
Token efficiency 变重要	高质量数据边际变稀缺	数据改写和优化器是否稳定。
开源有战略意义	生态验证和开发者扩散很强	商业捕获和维护成本。
架构效率不能牺牲智商	长上下文成本必须降	Linear/Sparse/Hybrid 的能力上限。
商业边界会重画	API、Agent 产品、生态互相牵制	谁掌握用户入口和反馈数据。

把 EP113 放进张小珺 AI 队列

EP119 讲 Attention 架构考古，EP115 讲 Agent 下半场理论，EP116 讲企业级 Agentic Model；EP113 则给出模型公司视角：K2 如何把基础模型、Agentic 能力和开源生态结合，并在数据墙后寻找新的 scaling 路线。

7.1 关键 takeaways

1. K2 的价值在于基础模型能力、编程/工具使用、Agentic 泛化和开源生态的组合。
2. Test-time scaling 是长思考与 Agent 工具使用的共同抽象。
3. Token efficiency 是数据墙时代的核心训练指标之一。
4. Linear Attention 等长上下文架构要同时看效率和“智商”风险。
5. 开源、API、Agent 产品和基座模型边界，是模型公司商业化的关键问题。

7.2 与前后几集的关系

节目	主题	与 EP113 的连接
EP119	Kimi Linear、DeepSeek、Mini-Max 注意力架构	解释 K2 背后的 Attention/长上下文架构取舍。
EP115	Agent 下半场、reward、interface	给出 Agentic LLM 的理论框架。
EP116	企业级 Agentic Model	展示 Agentic 能力在 ToB 私有数据场景的落地版本。
EP118	VLA、Agent OS、物理世界入口	把“模型长出手”从数字世界推向物理世界。

7.3 开放问题

前面给出结论，本节保留真正没有定论的部分。它们不是结尾装饰，而是 K2 之后模型公司和 Agent 产品公司都要继续回答的问题。

1. 数据墙之后，FLOPs scaling、RL feedback 和架构效率会怎样重新分配预算？
2. Linear Attention、Sparse Attention、Hybrid Attention 哪条路线能在能力和成本之间胜出？
3. 开源模型如何在生态扩散和商业捕获之间找到平衡？
4. Agentic LLM 的泛化应该如何评测，特别是在未知工具和未知任务上？
5. 基座模型公司会吞掉 Agent 应用，还是 Agent 应用会反过来定义基座模型需求？

开放问题的价值

这些问题不是附录，而是模型公司真实决策的核心。K2 的意义也正在于它把这些问题同时暴露出来：模型、数据、工具、生态、商业和组织没有一个可以单独解决。

术语对照附表

术语	可记忆解释	在本期中的位置
缸中之脑	只有认知、没有行动接口的模型	导读和 Agentic LLM 主线。
万能构造器	能利用工具改变世界的行动者	Agent 能力和人类类比。
数据墙	高质量训练数据边际变稀缺	Scaling 与数据飞轮讨论。
智商风险	降成本架构可能损害模型能力	Linear Attention 取舍。
RL 管理	用目标和反馈塑造团队探索	创始人组织隐喻。

7.4 拓展阅读

- 对 Attention 架构取舍感兴趣，可对照 EP119 的 Kimi Linear / MiniMax M2 / DeepSeek 注意力综述。
- 对 Agent 理论感兴趣，可对照 EP115 姚顺雨访谈，理解 reward 、multi-agent 和 interface 。
- 对企业落地感兴趣，可对照 EP116 吴明辉访谈，观察 Agentic Model 在 ToB 私有数据场景中的形态。